

Sri Lanka Institute of Information Technology



IT2130 – Operating Systems and System Administration
Year 2, Semester 2- 2026

Practical Answer Submission

Practical Sheet No: 08

Student ID	IT24102099
Student Name	Mathuppriya Naguleswaran
Campus/ Center name	SLIIT – Northern UNI
Specialization	Software Engineering
Batch No	1

The use of Shared memory in C language.

Activity 1

The first screenshot shows the source code for Lab8_A1.c in the nano editor. The code includes headers for stdio, sys/ipc, sys/shm, and string. The main function sets a key, creates shared memory with shmget, attaches it with shmat, writes 'Hello from shared memory!' and 'Data written to shared memory!' using sprintf and printf, detaches it with shmdt, and returns 0.

```
GNU nano 7.2 Lab8_A1.c
#include <stdio.h>
#include <sys/ipc.h>
#include <sys/shm.h>
#include <string.h>

int main() {
    key_t key = 1234;
    int shmid;
    char* ptr;

    shmid = shmget(key, 4096, 0666 | IPC_CREAT);
    ptr = (char*)shmat(shmid, NULL, 0);
    sprintf(ptr, "Hello from shared memory!");
    printf("Data written to shared memory!");
    shmdt(ptr);
    return 0;
}
```

The second screenshot shows the terminal output after compiling and running the program. The commands used are nano Lab8_A1.c, gcc -o Lab8_A1 Lab8_A1.c, and ./Lab8_A1. The output shows 'Data written to shared memory!' being printed twice.

```
mathuppriya-nakuleswaran@mathuppriya-nakuleswaran-VirtualBox: ~
mathuppriya-nakuleswaran@mathuppriya-nakuleswaran-VirtualBox:~$ nano Lab8_A1.c
mathuppriya-nakuleswaran@mathuppriya-nakuleswaran-VirtualBox:~$ gcc -o Lab8_A1 Lab8_A1.c
mathuppriya-nakuleswaran@mathuppriya-nakuleswaran-VirtualBox:~$ ./Lab8_A1
Data written to shared memory!
Data written to shared memory!
mathuppriya-nakuleswaran@mathuppriya-nakuleswaran-VirtualBox:~$
```

Figure 8.1 : Create shared memory and read & write.

Line No	Code	Explanation
1	#include <stdio.h>	used for input and output functions like - printf() - sprintf()
2	#include <sys/ipc.h>	It is needed for IPC features such as shared memory keys.

3	#include <sys/shm.h>	It contains the shared memory system calls: ▪ shmget() ▪ shmat() ▪ shmdt() ▪ shmctl()
4	#include <string.h>	This is used for string handling. In this code it is not heavily used, but it is commonly included when working with strings.
5	int main() {	The program starts here.
6	key_t key = 1234;	It creates a key for the shared memory. To identify the shared memory.
7	int shmid;	This variable stores the shared memory ID . After creating or finding shared memory, the system gives back an ID. That ID is stored in shmid.
8	char *ptr;	This is a pointer to char . It will point to the shared memory after attaching it. Since the data is text, char * is used.
9	shmid = shmget(key, 4096, 0666 IPC_CREAT);	shmget() : get shared memory ▪ It either: creates a new shared memory segment, or gets an existing one. Parameters: key → identifies the shared memory 4096 → size in bytes 0666 → read/write permission IPC_CREAT → create the memory segment if it does not already exist. The shared memory segment is created, and its ID is stored in shmid.

10	<pre>ptr = (char *) shmat(shmid, NULL, 0);</pre>	This attaches the shared memory to the process shmat() : shared memory attach (Now the process can use the memory.) Parameters: shmid → shared memory ID NULL → let system choose the address 0 → normal access mode ptr now points to the shared memory block. So now, ptr can be used like a normal string variable in memory.
11	<pre>sprintf(ptr, "Hello from shared memory!");</pre>	This writes the message into shared memory. sprintf() : write formatted data into a string Here: destination = ptr text = "Hello from shared memory!" So this line stores that message in the shared memory.
12	<pre>printf("Data written to shared memory\n");</pre>	This prints a normal output message on the screen. Here It tells the user that data was written successfully.
13	<pre>shmdt(ptr);</pre>	This detaches the shared memory from the process. shmdt() : shared memory detach The program is done using the shared memory, so it disconnects from it. This does not delete the shared memory. It only removes this process's connection to it
14.	<pre>return 0;</pre>	This ends the program successfully.

Activity 2

```
mathuppriya-nakuleswaran@mathuppriya-nakuleswaran-VirtualBox: ~
GNU nano 7.2 Lab8_A2.c
#include <stdio.h>
#include <sys/ipc.h>
#include <sys/shm.h>

int main() {
    key_t key = 1234;
    int shmid;
    char*ptr;

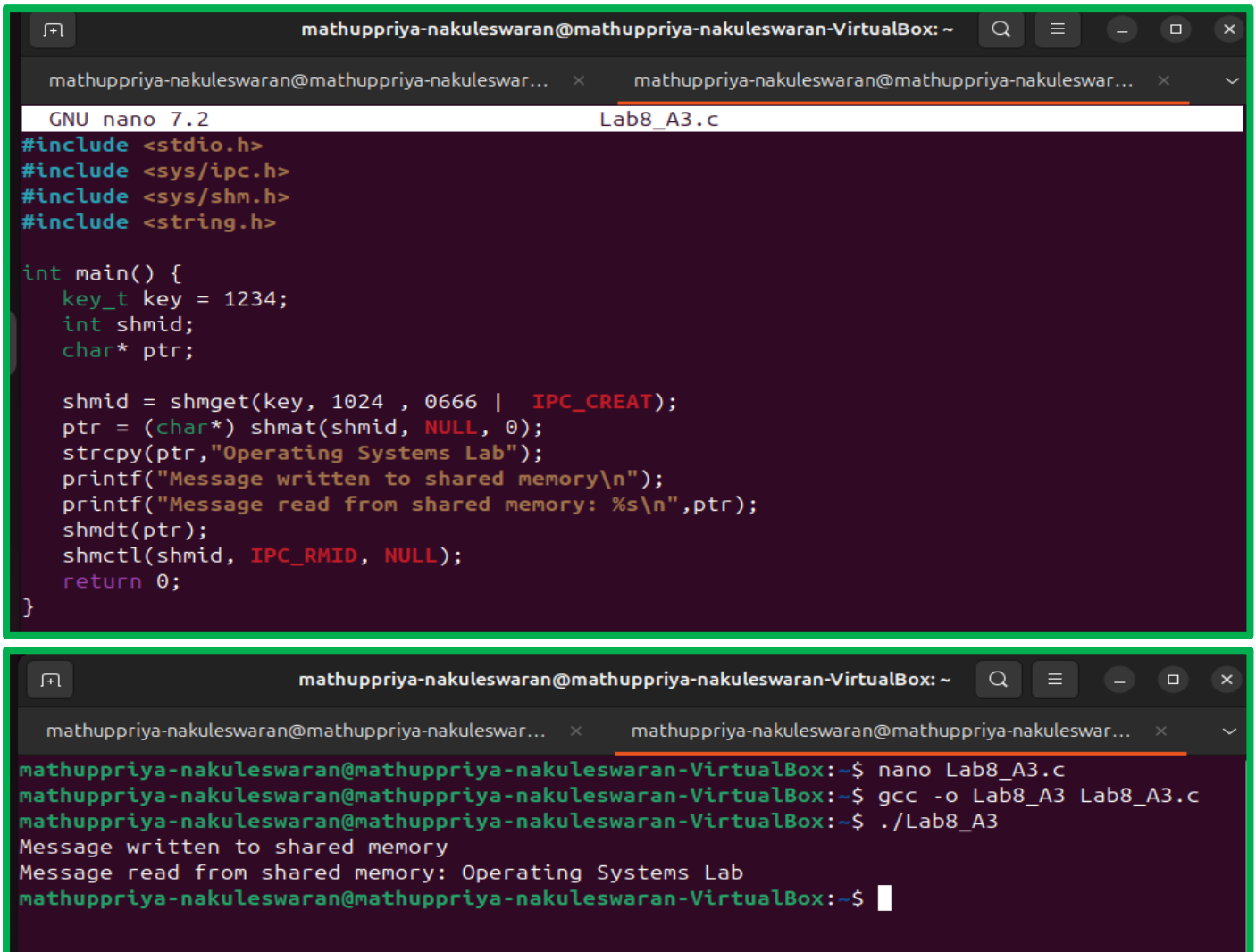
    shmid = shmget(key,4096,0666);
    ptr = (char*) shmat(shmid,NULL,0);
    printf("Read from shared memory : %s\n",ptr);
    shmdt(ptr);
    shmctl(shmid,IPC_RMID,NULL);
    return 0;
}

mathuppriya-nakuleswaran@mathuppriya-nakuleswaran-VirtualBox: ~$ nano Lab8_A2.c
mathuppriya-nakuleswaran@mathuppriya-nakuleswaran-VirtualBox:~$ gcc -o Lab8_A2 Lab8_A2.c
mathuppriya-nakuleswaran@mathuppriya-nakuleswaran-VirtualBox:~$ ./Lab8_A2
Read from shared memory : Hello from shared memory!
mathuppriya-nakuleswaran@mathuppriya-nakuleswaran-VirtualBox:~$
```

Line No	Code	Explanation
1	#include <stdio.h>	need for printing output
2	#include <sys/ipc.h>	need for IPC
3	#include <sys/shm.h>	need for shared memory functions
4	int main() {	Program starts here.
5	key_t key = 1234;	- Same key as the first program. - important because the reader program must use the same key to access the same shared memory.
6	int shmid;	shmid stores shared memory ID.
7	char *ptr;	ptr points to the attached shared memory
8	shmid = shmget(key, 4096, 0666);	This gets the existing shared memory segment.

		- Here there is no IPC_CREAT : Because this program is only trying to access existing shared memory, not create a new one. - If the memory is already created by the first program, this line gets its ID.
9	ptr = (char *) shmat(shmid, NULL, 0);	Attach the shared memory to this process. Now ptr points to the shared memory where the message is stored.
10	printf("Read from shared memory: %s\n", ptr);	This prints the contents stored in shared memory. Since ptr points to the message, the output will be something like: Read from shared memory: Hello from shared memory!
11	shmdt(ptr);	Detach the shared memory from this process.
12	shmctl(shmid, IPC_RMID, NULL);	This line removes the shared memory segment. shmctl() : control operation on shared memory Parameters: shmid → shared memory ID IPC_RMID → remove the shared memory NULL → no extra structure needed So this line deletes the shared memory from the system. Important difference: shmdt(ptr) → detach only shmctl(..., IPC_RMID, NULL) → completely remove shared memory
13	return 0;	End of program.

Activity 3



```
mathuppriya-nakuleswaran@mathuppriya-nakuleswaran-VirtualBox: ~
GNU nano 7.2 Lab8_A3.c
#include <stdio.h>
#include <sys/ipc.h>
#include <sys/shm.h>
#include <string.h>

int main() {
    key_t key = 1234;
    int shmid;
    char* ptr;

    shmid = shmget(key, 1024 , 0666 | IPC_CREAT);
    ptr = (char*) shmat(shmid, NULL, 0);
    strcpy(ptr,"Operating Systems Lab");
    printf("Message written to shared memory\n");
    printf("Message read from shared memory: %s\n",ptr);
    shmdt(ptr);
    shmctl(shmid, IPC_RMID, NULL);
    return 0;
}
```

```
mathuppriya-nakuleswaran@mathuppriya-nakuleswaran-VirtualBox: ~$ nano Lab8_A3.c
mathuppriya-nakuleswaran@mathuppriya-nakuleswaran-VirtualBox:~$ gcc -o Lab8_A3 Lab8_A3.c
mathuppriya-nakuleswaran@mathuppriya-nakuleswaran-VirtualBox:~$ ./Lab8_A3
Message written to shared memory
Message read from shared memory: Operating Systems Lab
mathuppriya-nakuleswaran@mathuppriya-nakuleswaran-VirtualBox:~$
```

Figure 8.3 : To implement a shared memory using system calls & Create shared memory & write a message into it, Then read the message from the shared memory & print it.